SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I (P.Veeraswami /192311253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Veeraswani

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWERS

Answer 1: Online Banking – Fund Transfer Boundary Failures

1. Problem Understanding & Context Analysis

Failures occur mostly at the minimum and maximum transfer limits. This shows that the current suite checks typical amounts only and never exercises the edges of the input domain, where off-by-one errors such as '>' used instead of '>=' are most common. Random testing makes this worse: if the valid range is ₹1 to ₹1,00,000, a randomly picked amount hits an exact boundary with a probability of roughly 1 in 1,00,000. Assumed business rule for this answer: minimum ₹1, maximum ₹1,00,000 per transaction, and amount must not exceed the available balance.

2. Application of Concepts (ECP + BVA)

Equivalence Class Partitioning (ECP) for the 'amount' field and related conditions:

Class	Description	Representative value	Expected result
EC1 (invalid)	Amount < minimum (zero / negative)	-500	Rejected with error message
EC2 (valid)	Minimum ≤ amount ≤ maximum and ≤ balance	25,000	Transfer successful
EC3 (invalid)	Amount > maximum limit	2,00,000	Rejected: limit exceeded
EC4 (invalid)	Amount within limit but > available balance	Balance 10,000; amount 20,000	Rejected: insufficient funds
EC5 (invalid)	Non-numeric / blank / more than 2 decimals	'abc', blank, 10.555	Rejected: invalid format

Boundary Value Analysis (BVA) on the amount limits (min = 1, max = 1,00,000):

TC	Amount	Boundary point	Expected result
TC1	0.99	Just below minimum	Rejected
TC2	1.00	Minimum	Accepted
TC3	1.01	Just above minimum	Accepted

TC	Amount	Boundary point	Expected result
TC4	99,999.99	Just below maximum	Accepted
TC5	1,00,000.00	Maximum	Accepted
TC6	1,00,000.01	Just above maximum	Rejected
TC7	Amount = balance exactly	Balance boundary	Accepted; balance becomes 0 (or minimum-balance rule applied)
TC8	Amount = balance + 0.01	Balance boundary	Rejected: insufficient funds

3. Analytical & Decision-Making Skills

Current strategy evaluation: the suite has no systematic coverage of the input domain, no negative cases and no traceability to the limit rules, so its defect-detection power at boundaries is close to zero.

Random testing decision: Replace it as the primary technique and retain it only as a supplementary one. It cannot guarantee that boundaries, invalid classes or rule combinations are tested, and its results are not repeatable unless the seed is recorded. It is still useful for fuzzing unexpected inputs and for operational-profile style testing after the systematic tests pass, because it can reveal unanticipated failures that ECP/BVA do not predict.

4. Solution Design & Approach

- Step 1 – Derive input conditions from the requirement (amount, balance, account status, daily limit).
- Step 2 – Apply ECP to get one valid and all invalid classes; apply BVA to every numeric limit (min, max, balance, daily cumulative limit).
- Step 3 – Use a decision table for combined conditions (amount valid × balance sufficient × account active × daily limit not exceeded) so that interactions are tested, not only single fields.
- Step 4 – Automate the suite for regression and run it on every build; add seeded random/fuzz tests as a final layer.
- Step 5 – Track coverage as 100% of equivalence classes and 100% of boundary points, and review defects by class to refine the suite.

5. Clarity, Justification & Presentation

ECP reduces an effectively unlimited amount domain to a handful of classes, and BVA targets the exact values where the observed failures occur. Together they give maximum defect detection with a small, repeatable and traceable test set (about 13 tests instead of thousands of random ones). The hybrid of structured tests plus limited random testing is therefore the optimal approach.

Answer 2: Healthcare Module – Post-Deployment Defects due to Poor Coverage

1. Problem Understanding & Context Analysis

The module passed testing but failed in production, which indicates that the test adequacy criterion was weak (typically 'all main-flow requirements covered') and that exceptional paths such as invalid data, allergy alerts and dose limits were never executed. For a conceptual illustration, consider a dosage module with the following logic: (1) read age, weight, allergy; (2) if data invalid, show error; (3) if allergy, raise alert and stop; (4) if age < 12 compute paediatric dose, else adult dose; (5) if dose > maximum, cap it; (6) output dose.

2. Application of Concepts (Control Flow Graph)

Nodes: N1 input; N2 decision 'data valid?'; N3 error; N4 decision 'allergy?'; N5 alert; N6 decision 'age < 12?'; N7 paediatric dose; N8 adult dose; N9 decision 'dose > max?'; N10 cap dose; N11 output/exit. Edges (14): 1-2, 2-3, 2-4, 3-11, 4-5, 4-6, 5-11, 6-7, 6-8, 7-9, 8-9, 9-10, 9-11, 10-11. Cyclomatic complexity V(G) = E − N + 2 = 14 − 11 + 2 = 5 (equal to 4 decision nodes + 1).

Path	Route	Meaning	Covered by old suite?
P1	1-2-3-11	Invalid data rejected	No
P2	1-2-4-5-11	Allergy alert	No
P3	1-2-4-6-7-9-11	Child, normal dose	Partly

Path	Route	Meaning	Covered by old suite?
P4	1-2-4-6-8-9-11	Adult, normal dose	Yes (only path tested)
P5	1-2-4-6-8-9-10-11	Dose exceeds maximum, capped	No

3. Analytical & Decision-Making Skills

Coverage assessment (illustrative): the old suite used only valid, non-allergic adult cases (path P4). Statement coverage = 7 of 11 nodes ≈ 64%; branch coverage = 6 of 14 edges ≈ 43%; independent-path coverage = 1 of 5 = 20%. The untested paths contain exactly the safety-critical logic (error handling, allergy and overdose protection), which explains the post-deployment failures. Statement coverage alone would also have been misleading because one statement can hide several untested outcomes.

4. Solution Design & Approach

Test	Input	Path / edges covered	Expected outcome
T1	age = -1	P1 (2→3)	Error message, no dose
T2	Valid data, allergy = Yes	P2 (4→5)	Allergy alert, no dose
T3	age = 8, weight = 25, no allergy	P3 (6→7, 9→11)	Paediatric dose within limit
T4	age = 40, weight = 70	P4 (6→8, 9→11)	Adult dose within limit
T5	age = 40, weight = 200 (very high)	P5 (9→10)	Dose capped at maximum

These five tests give 100% statement, 100% branch and 100% basis-path coverage (T1, T2, T3 and T5 alone already give 100% branch coverage). Black-box BVA tests (age 11/12, weight at limits) are added to cover the decision boundaries, and condition coverage is applied to the compound condition in N2.

5. Clarity, Justification & Presentation

Because every decision outcome and every independent path is executed, no logic is left unexecuted, which reduces the probability of latent defects. Safety-critical paths are verified explicitly, so reliability increases and risk of patient harm decreases. Coverage is measurable, so the team can show objective exit criteria (100% branch, all basis paths) instead of relying on judgement.

Answer 3: E-Commerce – Failures in Input Combinations (Price, Quantity, Discount)

1. Problem Understanding & Context Analysis

Errors appear only for specific combinations of price, quantity and discount. The existing approach tests each field separately with a few typical values, so interaction faults (for example a 100% discount with a minimum price giving a zero or negative total, or maximum price × maximum quantity overflowing) go undetected. Assumed rules: price 0.01 – 99,999.99; quantity 1 – 100; discount 0 – 100%; total = price × quantity × (1 − discount/100).

2. Application of Concepts

Field	Valid class	Invalid classes	Boundary values
Price	0.01 – 99,999.99	≤ 0; > 99,999.99; non-numeric	0, 0.01, 0.02, 99,999.98, 99,999.99, 1,00,000
Quantity	1 – 100	< 1; > 100 (stock); decimal	0, 1, 2, 99, 100, 101
Discount %	0 – 100	< 0; > 100	-1, 0, 1, 99, 100, 101

Decision table (validity of inputs):

Rule	Price valid	Qty valid	Discount valid	Action
R1	Y	Y	Y	Calculate total
R2	N	Y	Y	Error: invalid price
R3	Y	N	Y	Error: invalid quantity
R4	Y	Y	N	Error: invalid discount

Rule	Price valid	Qty valid	Discount valid	Action
R5	N	N	Any	Error: multiple invalid inputs, all messages shown
R6	N	Y	N	Error: price and discount
R7	Y	N	N	Error: quantity and discount
R8	N	N	N	Error: all fields invalid

Because 3 fields × 3 levels give 27 combinations, a pairwise (orthogonal array L9) design is used for the valid boundary/nominal values. Every pair of values across any two fields appears at least once in 9 tests:

TC	Price	Qty	Discount %	Expected total
1	0.01	1	0	0.01
2	0.01	50	50	0.25
3	0.01	100	100	0.00
4	500	1	50	250.00
5	500	50	100	0.00
6	500	100	0	50,000.00
7	99,999.99	1	100	0.00
8	99,999.99	50	0	49,99,999.50
9	99,999.99	100	50	49,99,999.50

3. Analytical & Decision-Making Skills

Limitations of the current approach: it is single-field, ignores interactions, has no negative tests and has no coverage measure. Exhaustive testing of every combination is impractical, whereas ECP + BVA alone would still miss interaction defects. The hybrid of ECP, BVA, decision table and pairwise selection is chosen because it covers each field's boundaries and the combinations of them with a small number of tests.

4. Solution Design & Approach

- Valid combination tests: the 9 pairwise tests above (expected value calculated from the formula).
- Single-fault negative tests (one invalid field, others valid): price 0, price 1,00,000, quantity 0, quantity 101, discount -1, discount 101 (6 tests).
- Multiple-fault tests from decision-table rules R5 to R8 (4 tests) to confirm that one error does not mask another.
- Rounding and overflow checks: a rounding case such as price 0.01 × 3 at 33.33% discount, and the maximum total value.

5. Clarity, Justification & Presentation

The suite has about 19 tests instead of an unbounded input space, and it covers all equivalence classes, all boundaries, all decision-table rules and all pairs of field values. Most interaction defects are caused by two parameters, so pairwise coverage gives high defect detection while keeping the suite compact, and the decision table gives completeness for validation logic.

Answer 4: Nested Conditions and Loops – Partial Path Coverage

1. Problem Understanding & Context Analysis

The team reports only partial path coverage in a module with nested conditions and loops. In such code the number of distinct execution paths grows very quickly with every decision and loop, so testers who select data by intuition cover only the common route. For illustration, consider: for each element x[i] of n elements: if x[i] > 0 then (if x[i] is even then action A else action B) else action C.

2. Application of Concepts (CFG and Independent Paths)

Nodes: N1 initialise; N2 loop test 'i < n'; N3 decision 'x > 0'; N4 decision 'x even'; N5 action A; N6 action B; N7 action C; N8 i++; N9 output. Edges (11): 1-2, 2-3, 2-9, 3-4, 3-7, 4-5, 4-6, 5-8, 6-8, 7-8, 8-2. V(G) = 11 − 9 + 2 = 4 (3 predicates + 1).

Basis path	Route	Test data (x[])	Expected
BP1	1-2-9	n = 0 (empty)	No action, output default
BP2	1-2-3-4-5-8-2-9	[4]	Action A
BP3	1-2-3-4-6-8-2-9	[3]	Action B
BP4	1-2-3-7-8-2-9	[-1]	Action C

3. Analytical & Decision-Making Skills

Gaps identified: (a) the CFG probably omitted the zero-iteration edge and the loop back-edge, so the empty-input path was never considered; (b) conditions with compound expressions or early exits (break/continue) were drawn as one node, hiding edges; (c) only one loop iteration was tested, so paths that combine different branches in successive iterations were never run; (d) independent paths were not derived from V(G), so the team had no target for the minimum number of tests.

Impact: every independent path that is not executed leaves a behaviour unverified, and defects hide in exactly these rarely used paths, for example the negative-value branch or an empty collection. Statement coverage might still be high while branch and path coverage remain low.

4. Solution Design & Approach

- Construct a complete CFG (include loop back-edges, zero-iteration exit, and each condition of compound predicates).
- Compute V(G) = 4 and derive the 4 basis paths above as the minimum set; derive test data by solving the path predicates.
- Add loop-boundary tests: 0, 1, 2, typical, n−1, n and n+1 iterations (for the maximum n allowed).
- Add multi-iteration combination tests such as [4, 3, -1], which executes A, B and C in one run, and sequences like [-1, 4] to check that state from earlier iterations does not affect later ones.
- Prioritise critical paths (those that modify data or involve money/safety) by risk, and verify coverage with a tool (statement, branch, basis path).

5. Clarity, Justification & Presentation

Basis path testing guarantees that every decision outcome is executed at least once (100% branch coverage, which subsumes statement coverage), and loop testing adds the iteration counts that path testing alone does not force. Together they give the strongest practical coverage for the effort, since testing all possible paths is infeasible because of loops. Expected outcome: the untested behaviours are exercised, defects in empty-input, negative-input and multi-iteration behaviour are found early, and coverage can be reported objectively.

Answer 5: Financial Application – Defects Despite Black-box and White-box Testing

1. Problem Understanding & Context Analysis

Both techniques are used but defects still reach production. This normally means that the techniques were applied in isolation, with no link between them, and that coverage metrics were applied to the wrong things: black-box tests derived from requirements without measuring how much code they exercise, and white-box tests written to hit a coverage number rather than to find faults.

2. Application of Concepts

Aspect	Observed weakness (typical)	Consequence
Test design integration	Black-box and white-box teams work separately; no gap analysis	Duplicate tests for common paths, untested rare paths

Aspect	Observed weakness (typical)	Consequence
Adequacy criteria	Statement coverage or 'all requirements tested' used as exit criterion	Branch, condition and boundary defects remain
Black-box depth	ECP without BVA, or valid classes only	Boundary and invalid-input defects missed
White-box depth	No CFG / independent paths for complex calculation code	Untested paths in interest, fee and rounding logic
Traceability	No mapping requirement → test → code	Cannot show what remains uncovered

3. Analytical & Decision-Making Skills

Evaluation: coverage metrics were not used as feedback. A high statement coverage figure says nothing about untested decision outcomes, boundary values or compound conditions, and black-box coverage was never measured against the code. The two techniques are complementary, since black-box testing finds missing or wrongly implemented requirements and white-box testing finds unexecuted code and logic errors, so they must be combined in a single cycle.

4. Solution Design & Approach – Hybrid Strategy

Step	Activity	Technique	Exit criterion
1	Derive functional tests from requirements	ECP, BVA, decision tables	100% classes and boundaries covered
2	Build CFG for critical modules; compute V(G)	CFG, cyclomatic complexity	CFG reviewed
3	Execute step-1 tests with a coverage tool	Coverage measurement	Statement and branch % reported
4	Gap analysis: list uncovered branches and paths	Basis path testing	Every gap has a new test or a justified exclusion
5	Add condition-level tests for compound decisions	Condition / MC/DC for critical logic	100% branch; MC/DC on money-critical decisions
6	Evaluate test quality	Mutation testing, review of defect escapes	Mutation score above agreed target
7	Regression and feedback loop	Automated regression, defect taxonomy	Each production defect adds a test

Worked example – transaction fee rule: fee = 0 for amount ≤ 1,000; 1% for 1,000 < amount ≤ 50,000; 0.5% for amount > 50,000. BVA tests: 1,000, 1,000.01, 50,000, 50,000.01. CFG with two decisions has V(G) = 3, so at least 3 basis paths are tested; the coverage report then shows whether any branch such as the rounding step remained unexecuted, and extra tests are added.

5. Clarity, Justification & Presentation

The hybrid approach gives better coverage because black-box tests ensure that all requirements and boundaries are checked, while white-box measurement shows which code those tests missed and path tests fill the gap. It prevents defects by using multiple adequacy criteria (class, boundary, branch, path, MC/DC) instead of one, by adding mutation testing to challenge the quality of the tests and by feeding every escaped defect back into the suite. Conclusion: integrate, measure, close the gaps, and repeat.

Answer 6: HR Payroll Bonus – Errors at 5, 10 and 15 Years of Service

1. Problem Understanding & Context Analysis

Employees with exactly 5, 10 and 15 years receive incorrect payouts. The common cause is an incorrect relational operator at the band limits (for example 'years > 5' where the policy states '5 years or more'). Assumed rule: bonus band by service: Band 1: 0 to < 5 years; Band 2: 5 to < 10; Band 3: 10 to < 15; Band 4: 15 or more. The bonus percentage also depends on the performance rating (1 to 5). Because the defect appears only on the exact limit values, the existing test design evidently did not include them.

2. Application of Concepts (ECP + BVA)

Years of service class	Range	Type
EC-Y0	< 0	Invalid
EC-Y1	0 ≤ years < 5	Valid – Band 1
EC-Y2	5 ≤ years < 10	Valid – Band 2
EC-Y3	10 ≤ years < 15	Valid – Band 3
EC-Y4	15 ≤ years ≤ 50	Valid – Band 4
EC-Y5	> 50 or non-numeric	Invalid

Performance rating classes: invalid (< 1, > 5, blank), 1–2 (low), 3 (average), 4–5 (high). BVA test values for years: -1, 0, 1, 4, 5, 6, 9, 10, 11, 14, 15, 16, 50, 51. Representative test suite (rating fixed at 3 for the service-boundary tests, then varied):

TC	Years	Rating	Expected band / result
1	-1	3	Rejected: invalid service
2	0	3	Band 1
3	4	3	Band 1
4	5	3	Band 2 (previously faulty)
5	6	3	Band 2
6	9	3	Band 2
7	10	3	Band 3 (previously faulty)
8	11	3	Band 3
9	14	3	Band 3
10	15	3	Band 4 (previously faulty)
11	16	3	Band 4
12	50 / 51	3	Band 4 / rejected
13	5, 10, 15	1 and 5	Boundary service combined with minimum and maximum rating
14	10	0 / 6	Rejected: invalid rating

3. Analytical & Decision-Making Skills

Failure analysis: exact values 5, 10 and 15 fall on the boundary between two equivalence classes; a test suite built from typical values such as 3, 7, 12 and 20 years executes the correct band for each class and therefore passes while the limit defect remains. Test design evaluation: no BVA, no traceability from policy limits to test cases.

Why ECP and BVA are superior to exploratory testing here: (1) the bonus rules are deterministic and precisely specified, so the expected result for each input can be calculated and checked; (2) boundary values are known in advance and can be tested exhaustively, whereas exploratory testing depends on the tester's intuition and may never use exactly 5, 10 and 15; (3) the tests are repeatable, documented and auditable, which matters for payroll and compliance; (4) coverage can be measured (every class and boundary); (5) the suite can be automated for regression whenever policy changes. Exploratory testing remains useful as a complement for usability and unexpected scenarios, but not as the primary technique for financial calculations.

4. Solution Design & Approach

Combine ECP and BVA on years of service, ECP on ratings, then use a decision table of band × rating to verify the bonus percentage for every combination (4 bands × 3 rating classes = 12 rules, each tested at one boundary and one nominal value). Automate the suite and run it for every payroll rule change.

5. Clarity, Justification & Presentation

The redesigned suite of about 14 core tests plus the band × rating matrix targets exactly the failing values, ensures no payout error remains at any band limit, and gives the audit trail needed for financial software.

Answer 7: Laboratory Temperature Controller – Missed Alarm and MC/DC

1. Problem Understanding & Context Analysis

The alarm failed for a particular sensor-malfunction sequence. In safety-critical control software this usually results from a compound condition in which one atomic condition (here the sensor fault flag) was never shown to affect the decision on its own. Assumed logic (set-point 25 °C, deadband ±2 °C, valid range 5 to 45 °C, critical 40 °C):

- D1: if (sensorFault OR T < 5 OR T > 45) then ALARM and stop
- D2: else if (T < 23 AND heaterOK) then heater ON
- D3: else if (T > 27 AND coolerOK) then cooler ON; otherwise idle
- D4: if (T > 40 AND NOT coolerOn) then ALARM

2. Application of Concepts (Control Flow Graph)

Nodes: N1 read inputs; N2 D1; N3 alarm and stop; N4 D2; N5 heater ON; N6 D3; N7 cooler ON; N8 idle; N9 D4; N10 alarm; N11 exit. Edges (14): 1-2, 2-3, 2-4, 3-11, 4-5, 4-6, 5-9, 6-7, 6-8, 7-9, 8-9, 9-10, 9-11, 10-11. V(G) = 14 − 11 + 2 = 5, so five basis paths are required as a minimum.

3. Analytical & Decision-Making Skills

Gap in branch coverage: branch coverage only needs D1 to be true once and false once. A test with temperature 50 °C makes D1 true, so the branch is 'covered' although the sensorFault condition was never set to true while temperature was in range. A sensor that reports a plausible value but flags a fault (the malfunction sequence) therefore passed unnoticed. The same applies to the compound conditions in D2 to D4: branch coverage does not show that each atomic condition independently affects the outcome.

4. Solution Design & Approach – MC/DC Test Set

MC/DC requires each atomic condition to be shown to independently change the decision outcome, needing n + 1 tests for n conditions instead of 2^n. The following 10 tests (heaterOK and coolerOK = true unless stated) achieve 100% MC/DC for D1 to D4 and 100% branch coverage:

TC	sensorFault	T (°C)	heaterOK	coolerOK	Expected outcome	MC/DC role
T1	F	20	T	T	Heater ON	D2 baseline (A=T,B=T)
T2	T	25	T	T	ALARM	D1: sensorFault alone changes outcome
T3	F	4	T	T	ALARM	D1: T < 5 alone
T4	F	46	T	T	ALARM	D1: T > 45 alone
T5	F	25	T	T	Idle, no alarm	D1 all false; D2/D3 A=F
T6	F	20	F	T	No heater; idle	D2: heaterOK alone (vs T1)
T7	F	30	T	T	Cooler ON	D3 baseline
T8	F	30	T	F	Idle	D3: coolerOK alone (vs T7); D4: A=F
T9	F	42	T	T	Cooler ON, no alarm	D4: B false (vs T10)
T10	F	42	T	F	ALARM (high temperature, no cooling)	D4: both conditions true

Independence pairs: D1 – (T5,T2), (T5,T3), (T5,T4); D2 – (T5,T1) for T<23 and (T1,T6) for heaterOK; D3 – (T5,T7) and (T7,T8); D4 – (T8,T10) for T>40 and (T9,T10) for NOT coolerOn. The combination of T3 and T4 as a single case is infeasible (T cannot be both below 5 and above 45), so each is tested separately, which is a normal MC/DC treatment of coupled conditions.

5. Clarity, Justification & Presentation

In safety-critical systems a missed alarm is a hazard, so decision logic must be demonstrated, not assumed, to work for every condition. MC/DC detects masked conditions and logic mistakes (for example AND written as OR) that branch coverage misses, while needing only n + 1 tests per decision, which keeps the suite practical. It is the criterion required for the highest-criticality software levels in standards such as DO-178C, and applying it here

increases confidence that the alarm triggers under every defined combination of sensor fault, range and cooler availability. Hardware-in-the-loop tests of fault injection sequences should then confirm the timing behaviour.

Answer 8: Travel Portal – Promo Code Combinations Giving Negative Total

1. Problem Understanding & Context Analysis

Promo codes are tested one at a time, so the interaction between a percentage promo, a flat-amount promo and the user tier discount is never exercised. When the combined discount exceeds the fare, the total becomes negative. Testing in isolation is insufficient because the defect appears only when two or more discounts are applied together and the order of application and the floor at zero are not checked. Assumed rules: percentage promo 10%, Gold tier extra 5% (percentages are added), flat promo ₹500 applied after percentage discounts, and the total must never go below ₹0.

2. Application of Concepts (Decision Table)

Conditions: A = percentage promo applied; B = flat promo applied; C = Gold tier; D = discount ≥ fare (only relevant when B = Y).

Rule	A	B	C	D	Expected total (fare used in the test)
R1	N	N	N	–	Fare (₹1,000 → 1,000)
R2	N	N	Y	–	Fare × 0.95 (₹950)
R3	Y	N	N	–	Fare × 0.90 (₹900)
R4	Y	N	Y	–	Fare × 0.85 (₹850)
R5	N	Y	N	N	Fare − 500 (₹500)
R6	N	Y	N	Y	Floor at 0 (fare ₹400 → 0)
R7	N	Y	Y	N	Fare × 0.95 − 500 (₹450)
R8	N	Y	Y	Y	0 (fare ₹400 → 0)
R9	Y	Y	N	N	Fare × 0.90 − 500 (₹400)
R10	Y	Y	N	Y	0 (fare ₹400 → 0)
R11	Y	Y	Y	N	Fare × 0.85 − 500 (₹350)
R12	Y	Y	Y	Y	0 (fare ₹400 → 0)

3. Analytical & Decision-Making Skills

Full coverage of the three conditions plus the negative check is 12 effective rules (16 combinations before merging don't-care cells). Isolation testing covers only R1, R2, R3 and R5, which is 4 of 12 rules, and misses every rule with stacked discounts. Redundancy analysis: rules R6, R8, R10 and R12 exercise the same floor-to-zero logic; testing the minimum stack (R6) and the maximum stack (R12) is enough for the floor, which removes two tests without losing a distinct logic path.

4. Solution Design & Approach – Optimised Test Set

Group	Tests	Count
Valid stacking (no floor)	R1, R2, R3, R4, R5, R7, R9, R11	8
Floor to zero	R6 (single flat promo), R12 (maximum stacking)	2
Boundary of the floor (BVA)	Fare = ₹500.00 with flat promo → ₹0.00; fare = ₹500.01 → ₹0.01	2
Negative / invalid	Invalid code; expired code; same code applied twice (must be rejected or ignored)	3
Order of application	Flat promo applied before percentage promo (must give the same result as specified rule)	1

Total: 16 tests, instead of exhaustive testing of every price × code × tier combination. Cause-effect graphing yields the same 12 rules, because the causes (A, B, C, D) combine through AND/OR into the effect 'total computed' and the constraint 'total ≥ 0'.

5. Clarity, Justification & Presentation

The decision table is complete, so every logical combination is considered, and it is optimised, since equivalent rules are merged and only representative cases remain. It detects the negative-total defect directly (R6, R12 and the boundary tests), and it provides a clear specification for developers. Expected outcome: maximum defect detection with minimum redundancy.

Answer 9: Database Migration Tool – Infinite Loops despite High Statement Coverage

1. Problem Understanding & Context Analysis

The tool validates records through nested loops and its team reports high statement coverage, yet it crashes in production with infinite loops. For illustration: outer loop over records (i < n); inner loop over fields (j < m); for each field, while the value is invalid the tool attempts a repair and re-validates (retry loop) with no limit on retries.

2. Application of Concepts (CFG and Basis Paths)

Nodes: N1 i = 0; N2 outer test (i < n); N3 j = 0; N4 inner test (j < m); N5 decision 'field invalid?'; N6 repair; N7 j++; N8 i++; N9 exit. Edges (11): 1-2, 2-3, 2-9, 3-4, 4-5, 4-8, 5-6, 5-7, 6-5, 7-4, 8-2. V(G) = 11 − 9 + 2 = 4.

Basis path	Route	Scenario
BP1	1-2-9	No records (n = 0)
BP2	1-2-3-4-8-2-9	Record with zero fields (m = 0)
BP3	1-2-3-4-5-7-4-8-2-9	All fields valid
BP4	1-2-3-4-5-6-5-7-4-8-2-9	Field repaired once, then valid

3. Analytical & Decision-Making Skills

Why statement coverage was insufficient: it only demands that each statement is executed at least once. A test with a repairable invalid field executes the repair statement, the loop condition and the increments, so coverage looks complete. It never requires the repair loop to be executed repeatedly, the loop to be skipped, or the condition 'still invalid after repair' to be true again, so the non-terminating case (a record that cannot be repaired, such as a null key) is never run. Statement coverage also ignores decision outcomes, loop iteration counts and termination.

4. Solution Design & Approach

- Independent path tests BP1 to BP4 (minimum 4 tests as V(G) = 4).
- Loop boundary tests (Beizer's strategy) – outer loop: 0, 1, 2, typical, n−1, n, n+1 iterations; inner loop: 0, 1, 2, m−1, m, m+1; test inner loops at minimum values while the outer loop is at minimum, then increase the outer loop.
- Retry-loop tests: repairs needed 0, 1, 2 and k times (maximum allowed), and, most importantly, a record that can never be repaired. Expected behaviour: the tool stops after a retry limit, logs the record and continues or aborts cleanly instead of looping forever. Run these with a timeout watchdog.
- Data-driven tests: null values, empty strings, extremely long fields, duplicated records and the maximum table size.

5. Clarity, Justification & Presentation

Cyclomatic complexity V(G) = 4 gives the minimum number of independent paths to test, which is the lower bound on the test count and confirms that statement-level testing (often satisfiable with one or two tests) was inadequate. Because loops create an unbounded number of paths, basis paths alone cannot show termination, so loop boundary tests supply the missing evidence. Expected result: every decision outcome and loop count is exercised, the non-terminating case is detected before release, and a design correction (retry limit) is added.

Answer 10: Food Delivery Tracker – State Transition Testing

1. Problem Understanding & Context Analysis

The tracker moves through Ordered → Preparing → Out for Delivery → Delivered. When the driver cancels during delivery, the app freezes because the model has no defined transition for this event: the state 'Out for Delivery'

has an exit only for 'delivered'. The current black-box strategy tests screens and input values, not the sequences of events and the history of the order.

2. Application of Concepts (State Transition Testing)

Redesigned state model (states: Ordered, Preparing, Out for Delivery, Reassigning, Delivered, Cancelled/Refunded):

Current state	Event	Next state	Action
Ordered	Restaurant accepts	Preparing	Notify customer
Ordered	Customer cancels	Cancelled	Refund
Preparing	Food ready, driver picks up	Out for Delivery	Start tracking
Preparing	Driver cancels (before pickup)	Preparing (driver unassigned)	Assign new driver
Out for Delivery	Driver delivers	Delivered	Close order
Out for Delivery	Driver cancels	Reassigning	Notify customer, search driver
Reassigning	New driver accepts	Out for Delivery	Update tracker
Reassigning	No driver within timeout	Cancelled	Refund and notify
Delivered / Cancelled	Any event	No change (final state)	Reject event

Test scenarios (0-switch = single transitions; 1-switch = pairs of transitions):

TC	Scenario / event sequence	Expected result
S1	Ordered → Preparing → Out for Delivery → Delivered	Normal flow, each status shown
S2	Out for Delivery → driver cancels → Reassigning → new driver → Out for Delivery → Delivered	Tracker updates, no freeze
S3	Out for Delivery → driver cancels → Reassigning → timeout → Cancelled	Refund; app shows final status (was the reported defect)
S4	Preparing → driver cancels → driver reassigned → pickup	Order continues
S5	Driver cancels twice in a row (second driver cancels too)	Reassigns again, no duplicate state
S6	Reassigning → customer cancels	Cancelled and refunded once
S7	Invalid: Delivered → Preparing; Ordered → Delivered (skip)	Event rejected, state unchanged
S8	Network loss during Out for Delivery → driver cancel event arrives after reconnect	Tracker recovers to the correct state

3. Analytical & Decision-Making Skills

Why ECP and BVA miss this defect: they partition input values, but the failing behaviour depends on the system's history. The same input (the event 'driver cancel') produces different outcomes in each state, and the fault is a missing transition (a state with no outgoing edge for a legal event). A value-based approach has no notion of state, sequence or illegal transitions. State transition testing derives tests from the state diagram, so every state, every transition and every invalid state-event pair must be examined, which exposes dead ends such as 'stuck' states.

4. Solution Design & Approach

- Build the state table with all state × event pairs (including those marked invalid) and verify no valid event is left undefined.
- Coverage levels: all states, all transitions (minimum), all 1-switch sequences for critical paths (cancellation and reassignment), and all invalid transitions.
- Add timing tests (timeout expiry) and concurrency (customer and driver cancel at the same time).

5. Clarity, Justification & Presentation

State transition testing models the dynamic workflow explicitly, finds the stuck-state defect and similar logic gaps, and gives a measurable coverage target (100% transitions plus invalid event checks). It is the best complement to ECP and BVA, which remain useful for data fields such as address or tip amount.

Answer 11: Authentication Module – Brute-Force Vulnerability and Hybrid Testing

1. Problem Understanding & Context Analysis

The module combines password strength, OTP timing and IP location, but brute-force attacks succeeded. Functional black-box tests checked that valid and invalid credentials are accepted or rejected, but attacks depend on repeated attempts, state (failure counters, lockout) and the interaction between the factors, which these tests did not cover. White-box testing, if done, did not trace how the failure counter is updated on every path.

2. Application of Concepts (Formal Path Analysis)

Decisions: D1 account locked?; D2 password valid?; D3 failure count ≥ limit (e.g., 5)?; D4 OTP valid and within its time window?; D5 IP trusted? Five predicates give V(G) = 6, so six independent paths:

Path	Decision outcomes	Behaviour	Security question to test
P1	D1 = T	Reject, account locked	Is the lock checked before any credential check? Does it expire correctly?
P2	D1 = F, D2 = F, D3 = F	Reject, counter + 1	Is the counter incremented for every failure?
P3	D1 = F, D2 = F, D3 = T	Lock account	Does lockout occur at exactly 5 failures (4, 5, 6 attempts)?
P4	D1 = F, D2 = T, D4 = F	Reject, OTP failed	Does an OTP failure increment the counter? If not, 10^6 OTPs can be guessed
P5	D2 = T, D4 = T, D5 = F	Step-up verification / deny	Is access from an untrusted IP properly limited?
P6	D2 = T, D4 = T, D5 = T	Grant access	Is a session created only on this path?

3. Analytical & Decision-Making Skills

Black-box and white-box methods were used separately: black-box tests did not consider counters and sequences, while path coverage (if measured) did not include hostile data. Formal path analysis reveals which paths lack a security control (for example P4 with no counter update), but it does not suggest attack inputs. Error guessing contributes the attacker's knowledge: tests based on past vulnerabilities and tester experience. The two are therefore complementary and should be integrated.

4. Solution Design & Approach – Hybrid Strategy

Technique	Test ideas
ECP / BVA (baseline)	Password length min−1, min, max, max+1; OTP age 29 s, 30 s, 31 s; lockout at attempts 4, 5, 6; trusted / new / blocked IP
Formal path analysis	One test for each of P1 to P6, then verify the counter value after each path
Error guessing – brute force	Repeated wrong passwords from one IP; same attack rotating IPs or changing X-Forwarded-For header; attempts using several parallel sessions (race condition); OTP guessing 000000 to 999999 within the same window
Error guessing – logic abuse	Reuse of an already-used OTP; expired OTP; OTP of another user; skipping the OTP step by calling the final API directly; counter reset after a successful login on another device
Error guessing – input attacks	SQL injection (' OR 1=1 --), very long or Unicode passwords, null and empty fields; different response messages or timing for existing and non-existing users (user enumeration)

Expected result for every attack test: account is locked or throttled after the defined limit, generic error messages are shown, events are logged and no session is created.

5. Clarity, Justification & Presentation

Path analysis shows systematically where security checks exist and are missing, and error guessing supplies the creative attacker-style inputs for each path, so coverage is complete in structure and realistic in content. This integrated approach finds the brute-force weakness, reduces the attack surface and improves the security posture. The results should feed back as permanent regression tests and a threat model.

Answer 12: E-Learning Platform – Upgrade on the Expiry Day

1. Problem Understanding & Context Analysis

The platform has three tiers (Free, Basic, Premium) with access based on subscription dates and course type. Errors occur if the user upgrades on the day of expiration, which is a date boundary. Typical causes are comparison with '<' instead of '<=', and time-zone or end-of-day handling (a subscription ending 23:59:59 IST versus a server in UTC). The existing criteria tested only 'active' and 'expired' subscriptions.

2. Application of Concepts (ECP on Dates and Subscription Types)

Partition	Condition (E = expiry date)	Representative value	Expected behaviour
D1	Active: today < E	E − 10 days	Upgrade allowed, remaining period handled as specified
D1b	Boundary: today = E − 1	E − 1	Upgrade allowed
D2	Boundary: today = E (expiry day)	E, at 00:00 and at 23:59:59	Upgrade allowed, access continuous
D3	Boundary: today = E + 1	E + 1	Treated as expired; new subscription (no continuity)
D4	Expired for long time	E + 30 days	New purchase rules
D5 (invalid)	Null, malformed or start date > end date	'2026-13-45'	Error, no change of tier
D6	Free tier (no expiry date)	–	Date logic not applicable

Subscription type partitions: current tier Free / Basic / Premium; target tier Basic / Premium; valid upgrade (Free→Basic, Free→Premium, Basic→Premium), invalid (same tier or downgrade). Course type partitions: Free course, Basic course, Premium course.

3. Analytical & Decision-Making Skills

Adequacy evaluation: the criteria covered tier values but not date boundaries, state transitions on the boundary day or access after the upgrade, so the 'upgrade on expiry' combination was not a required test. Equivalence partitioning of dates reduces the date line to a few classes, and boundary values on the class edges expose the defect.

4. Solution Design & Approach

TC	Current tier	Upgrade to	Date class	Expected result
1	Basic	Premium	D1 (E − 10)	Success; new expiry computed per rule
2	Basic	Premium	E − 1	Success
3	Basic	Premium	E (00:00)	Success, no gap in access (defect case)
4	Basic	Premium	E (23:59:59)	Success (time-zone IST/UTC checked)
5	Basic	Premium	E + 1	Subscription expired; handled as new purchase
6	Free	Basic	Not applicable	Success; expiry date set
7	Free	Premium	Not applicable	Success; premium courses accessible
8	Premium	Basic	D1	Rejected or scheduled downgrade as specified
9	Basic	Basic	D1	Rejected: already on this tier
10	Basic	Premium	D5 invalid date	Error; tier unchanged
11	After TC 3	–	–	Access matrix: Free, Basic and Premium courses each accessible/blocked as per new tier, immediately

5. Clarity, Justification & Presentation

Every tier transition and every date class (including both sides of the expiry boundary and invalid dates) is covered, together with post-upgrade access, so no user transition class is left untested. The suite is small (about 11 tests)

yet complete in terms of equivalence classes and boundary values, and the defect on the expiry day is covered explicitly by TC 3 and 4.

Answer 13: Smart-Home Lighting Controller – Branch and Condition Coverage

1. Problem Understanding & Context Analysis

The controller uses a switch-case on the mode and Boolean expressions on sensor data. Assumed logic: switch(mode): AUTO – if ((lux < LOW AND occupied) OR manualBoost) level = HIGH; else if (lux > BRIGHT) level = OFF; else level = MEDIUM; MANUAL – level = userLevel; NIGHT – level = min(level, 20%); default – fault state, safe level.

2. Application of Concepts

Risks of relying solely on statement coverage: (1) one test per case executes every statement, but the false outcomes of decisions are never shown; (2) short-circuit evaluation means a condition such as 'occupied' may never be evaluated; (3) a missing break (fall-through) or an incorrect default may not be seen; (4) threshold behaviour (lux exactly equal to LOW or BRIGHT) is untested; (5) compound expression errors, for example AND replaced by OR, may pass because the executed statement is the same.

Branch coverage requires every switch case (including default) and each decision outcome true and false. Condition coverage requires each atomic condition (A = lux < LOW, B = occupied, C = manualBoost, D = lux > BRIGHT) to be true and false at least once.

TC	Mode	A	B	C	D	Decision outcome	Expected level
1	AUTO	T	T	F	–	(A∧B)∨C = T	HIGH
2	AUTO	F	T	F	F	F; D = F	MEDIUM
3	AUTO	T	F	F	F	F; B independent	MEDIUM
4	AUTO	F	F	T	–	T; C independent	HIGH
5	AUTO	F	F	F	T	F; D = T	OFF
6	AUTO	lux = LOW exactly	T	F	F	Boundary	MEDIUM (A false for '<')
7	MANUAL	–	–	–	–	Case MANUAL	User level
8	NIGHT	–	–	–	–	Case NIGHT; level capped	≤ 20%
9	Invalid (e.g., 7)	–	–	–	–	Default branch	Fault state / safe level

3. Analytical & Decision-Making Skills

The test set achieves 100% branch coverage (all four switch targets and both outcomes of each decision), and condition coverage (A, B, C and D each true and false). Missing logical paths identified: false outcome of the compound decision, default case, boundary lux values and the independent effect of each condition. For stronger confidence on the compound expression, MC/DC (tests 1 to 4) is recommended since condition coverage alone does not guarantee that decision outcomes both occur.

4. Solution Design & Approach

- Static analysis and code review for missing break statements and unreachable code.
- Branch + condition coverage (MC/DC for the dimming decision) with coverage measured by tool.
- Boundary tests around every sensor threshold, plus sensor faults: out-of-range value, stuck value, no data, noisy readings around a threshold (check hysteresis).
- Software-in-the-loop tests with simulated sensors, then hardware-in-the-loop tests of the same cases on the real controller.

5. Clarity, Justification & Presentation

Integration failures usually occur when an unexpected sensor value reaches a path that was never executed in software testing. By executing all branches, all atomic conditions, the default case and the threshold values in simulation, the software behaviour is verified before it drives real hardware, so faults such as flickering at

thresholds, wrong levels or no fail-safe state are found early and cheaply. This reduces the risk of hardware-software integration failures.

Answer 14: Logistics Shipping Cost – Zero and Negative Weight (Robustness Testing)

1. Problem Understanding & Context Analysis

The system calculates shipping cost from weight, destination zone and delivery speed. Zero and negative weights were accepted or caused wrong results, which means input validation was tested only with valid weights. Assumed valid range: 0.1 kg ≤ weight ≤ 30 kg, zones Z1 to Z4, speed Standard / Express / Overnight.

2. Application of Concepts (Robust BVA)

Normal BVA tests min, min+, nominal, max− and max. Robustness testing adds the values just outside the limits (min−, max+), plus extreme invalid values. For one bounded numeric variable that gives 7 values:

TC	Weight (kg)	Class	Expected result
1	-5	Invalid, far below	Rejected with message 'weight must be between 0.1 and 30 kg'
2	0	Invalid, zero	Rejected
3	0.09	min − 0.01	Rejected
4	0.1	Minimum	Accepted, correct cost
5	0.11	min + 0.01	Accepted
6	15	Nominal	Accepted
7	29.99	max − 0.01	Accepted
8	30	Maximum	Accepted
9	30.01	max + 0.01	Rejected (or freight rate if specified)
10	blank, 'abc', 5@, NaN, 1E+99	Invalid type / format / overflow	Rejected, no crash, no cost shown

Other fields: zone Z0, Z5, blank, lower-case 'z1'; speed 'Standard', 'Express', 'Overnight' and invalid 'Same-day' / blank. For n variables robust BVA gives 6n + 1 tests, applied one fault at a time with all other inputs at nominal values; here the three parameters give 6(3) + 1 = 19 tests at most (zone and speed are enumerations and use ECP instead of limits).

3. Analytical & Decision-Making Skills

Failure analysis: the validation logic was written for valid inputs and tested only with valid inputs, so rejection rules for zero, negative and non-numeric weights had never been verified. The result can be a free or negative shipping charge, because a formula such as rate × weight returns 0 or a negative value.

4. Solution Design & Approach

- Add robust BVA tests for all numeric limits and ECP invalid classes for zone and speed (above).
- Single-fault tests (one invalid input, others valid) to find which validation fails, then a few multi-fault tests (weight negative and zone invalid) to check that all errors are reported.
- Verify the error handling: clear message, no calculation, no incorrect shipment created, event logged, system continues.
- Automate in the regression suite for every pricing release.

5. Clarity, Justification & Presentation

Negative scenarios are necessary because real users, other systems and attackers do send invalid data. A robust system must reject it safely. Without negative tests the system can calculate wrong or negative charges (financial loss and abuse), crash or hang (availability), store corrupt data, or expose injection vulnerabilities. Robustness is a quality attribute in itself, and defects in input validation are cheap to find in testing but expensive in production.

Answer 15: Medical Imaging – Parallel Loops and Prime Data Size

1. Problem Understanding & Context Analysis

The software splits N data elements among T worker threads, processes each chunk in a loop and combines the partial results. The results are inconsistent only when N is a prime number. A prime N (greater than T) is never divisible by any thread count T > 1, so there is always a remainder, and the code path that assigns the leftover elements (last chunk, boundary of the partition, or tile/padding handling) is executed. A typical bug is chunk = N / T (integer division) with the remainder elements skipped or counted twice.

2. Application of Concepts (Basis Path Testing)

Conceptual logic: if (N < T) run sequentially; chunk = N / T; for t = 0 to T−1: start = t × chunk; if (t is the last thread) end = N (takes the remainder) else end = start + chunk; process(start, end); combine partial results. Decisions: D1 N < T; D2 loop over t; D3 last thread?; D4 remainder ≠ 0 in the last chunk. Four predicates give V(G) = 5, so five independent paths:

Basis path	Condition	Test data (N, T)	Expected result
BP1	D1 true: N < T	N = 7, T = 8	Sequential fallback; equals reference
BP2	T = 1, last chunk is the only chunk	N = 101, T = 1	Equals reference
BP3	T ≥ 2, remainder = 0	N = 100, T = 4	Equals reference
BP4	T ≥ 2, remainder ≠ 0 (prime N)	N = 101, T = 4	Equals reference; last chunk of 26 elements
BP5	Boundary N = T (chunk size 1)	N = 7, T = 7	Equals reference

3. Analytical & Decision-Making Skills

Evaluation of random testing: the input space of data sizes is very large and prime numbers are sparse (about 1 in ln N of all integers near N), so random sizes are mostly composite and often divisible by the thread count. The remainder path is therefore rarely executed, and random testing gives no coverage guarantee, no repeatability unless seeds are logged, and no argument of mathematical adequacy. In a medical application a failure that occurs only for specific sizes is exactly the kind a random approach misses.

4. Solution Design & Approach

- Control-flow analysis of the partitioning, work distribution and reduction code; basis path tests BP1 to BP5 as the minimum.
- Data-size partitions: primes (2, 3, 5, 7, 11, 13, 101, 1009, 65,537), powers of two (1024), composites (100), N = T − 1, T, T + 1, and N = 0 and 1.
- Thread-count variation: T = 1, 2, 4, 8 and a value larger than N; every size run with every thread count.
- Oracle: compare the parallel result with a trusted sequential implementation; for floating point use a defined tolerance because parallel reduction changes the order of addition. Add metamorphic tests (result independent of the number of threads and of chunk order).
- Concurrency checks: repeat each test many times, use a race-condition detector and deterministic seed, test shared-memory access at the chunk boundaries.

5. Clarity, Justification & Presentation

Basis path testing guarantees that every independent route through the partitioning logic, including the remainder path triggered by prime sizes, is executed. Targeted data-size classes and a reference oracle verify the numerical accuracy of the result for each class, and repeated multi-thread runs verify stability. The approach is systematic, repeatable and measurable, which is needed for a medical imaging product, whereas random testing leaves the critical cases to chance.